

The ABBA Numerical-Method Family

Duplicated phase space, projection, composition, and configuration axes

1 Purpose and model boundary

The ABBA model is a family of integrators built from the same explicit A – B – B – A map on two copies of a physical state. A public class selects the order and projection recipe; configuration controls select the projection equations, nonlinear solver, state extension, and optional physical energy tracking. These choices are configurations of a model, not additional model identities.

The family has four public method classes:

- `ABBA2Midpoint` and `ABBA2Implicit`;
- `ABBA4Implicit`, with one outer projection; and
- `ABBA6Implicit`.

Their focused theory documents live in the sibling model directories.

2 Duplicated-space ABBA map

Let $\dot{z} = f(t, z)$ with $z \in \mathbb{R}^m$. Duplicate the state as (u, v) and set $s = h/2$. One signed endpoint-time step beginning at t is

$$u_1 = u_0 + sf(t, v_0), \quad v_1 = v_0 + sf(t, u_1), \quad (1)$$

$$v_f = v_1 + sf(t + h, u_1), \quad u_f = u_1 + sf(t + h, v_f). \quad (2)$$

We denote this map by $\Psi_{h,t}(u_0, v_0) = (u_f, v_f)$. Its reverse is $\Psi_{-h,t+h}$, so the endpoint-time convention is essential for non-autonomous reversibility. When the split system is Hamiltonian, each line is a shear and the composition is symplectic on the duplicated space.

With

$$E = \begin{pmatrix} I \\ I \end{pmatrix}, \quad P = \frac{1}{2} \begin{pmatrix} I & I \end{pmatrix}, \quad G = \begin{pmatrix} I & -I \end{pmatrix}, \quad N = G^T,$$

Ez embeds the physical diagonal, P takes its arithmetic mean, and $G(u, v) = u - v$ measures copy separation.

3 Projection choices

The midpoint method accepts $P\Psi_{h,t}(Ez)$ directly. The implicit methods use Hairer's symmetric projection. In the reduced formulation a multiplier $\mu \in \mathbb{R}^m$ displaces the input and corrects the output:

$$\hat{Y} = Ez + N\mu, \quad M = \Psi_{h,t}(\hat{Y}), \quad (3)$$

$$r(\mu) = GM + 2\mu = 0, \quad z^+ = P(M + N\mu). \quad (4)$$

If $J_\Psi = D\Psi_{h,t}(\hat{Y})$, Newton uses

$$D_\mu r = GJ_\Psi N + 2I.$$

The simultaneous formulation solves for the corrected doubled output Y and μ together:

$$F(Y, \mu) = \begin{pmatrix} Y - N\mu - \Psi_{h,t}(Ez + N\mu) \\ GY \end{pmatrix} = 0.$$

Eliminating Y gives the reduced equation. Thus both formulations define the same ideal projected map when they converge to the same root. Newton uses the exact residual Jacobian; good Broyden replaces repeated Jacobian assembly by rank-one secant updates without changing the defining residual.

4 Canonical configuration space

ABBA2, ABBA4 and ABBA6 retain their distinct composition and projection placement. Energy tracking is optional and passive. Signed accepted shear-stage traces update the passive momentum independently of the spatial root solve. The common state conventions are specified below.

5 Order by symmetric composition

The projected ABBA2 map is a symmetric method of order two. Higher order is obtained by palindromic signed compositions. ABBA4 uses the triple jump (γ, δ, γ) with

$$\gamma = \frac{1}{2 - \sqrt[3]{2}}, \quad \delta = -\frac{\sqrt[3]{2}}{2 - \sqrt[3]{2}}, \quad 2\gamma + \delta = 1, \quad 2\gamma^3 + \delta^3 = 0.$$

ABBA6 uses seven Yoshida coefficients whose first, third, and fifth odd moments have the required cancellations. Negative coefficients are intrinsic to these real compositions, so the dynamics must be defined at all internal signed times.

6 Projection placement is part of the model

Method	ABBA maps	projection policy
ABBA2Midpoint	1	arithmetic mean
ABBA2Implicit	1	one symmetric projection
ABBA4Implicit	3	one outer projection around the composition
ABBA6Implicit	7	project after every signed map

The former three-projection fourth-order implementation has been removed. The current default has one projection, and legacy single-projection names refer to that same map.

7 Finite solves and geometric claims

Symmetry, formal order, and symplecticity concern the ideal map obtained from the exact projection root. A finite stopping tolerance and floating-point arithmetic perturb that map. Consequently, the implementation records iterations, residual evaluations, final residual norms, tolerances, and projection multipliers. Step observers expose the accepted map so measured Jacobian defects can be distinguished from ideal algebraic properties.

8 Implementation correspondence

The existing ABBA classes select their recipes in the `order*.py` modules under `src/methods/extended/`. Their common run operations live in `abba.py`; the spatial engine is shared with BM4.

The public names, constructor defaults and exports from `simulation` remain unchanged. Internal imports use the defining modules; old import adapters have been removed. The ordinary implicit path is `advance`, `solve_projection`, `compose`, then `direct_map` or `adjoint_map`. Arithmetic midpoint variants use one explicit average instead of a nonlinear projection. Adjoint stages evaluate at the start of their signed substep; direct stages evaluate at its end.

Newton and Broyden share the spatial equations and convergence services. Exact particle-block tangents, passive energy and optional observation use retained shear inputs; energy is evaluated only after accepting the spatial map, with the existing factor $1/2$ for physical κ . Time and momentum never enter the spatial root. The new internal family name does not enable the retired full time/momentum projection.

Current module responsibilities and diagrams are documented in `docs/models/extended/simulation/extended-simulation-architecture.md`.

9 Common integration lifecycle

Each integration initializes a fresh numerical instance through `IntegrationMethod.integrate`. Implicit orders share a family base. The fixed controller supplies accepted intervals and independent shadow samples; the coordinator owns progress, collection and event dispatch. Only sampled states and accepted metric rows are retained by the collector.

10 Four state formulations and passive energy tracking

For one planar particle the supported internal representations are

physical	$z = (x, y)$	$\mathbb{R}^2$,
physical with energy	(z, t, κ)	$\mathbb{R}^4$,
duplicated	$(u, v), \quad u, v \in \mathbb{R}^2$	$\mathbb{R}^4$,
duplicated with energy	(u, v, t, κ)	$\mathbb{R}^6$.

Classical methods use the physical rows; ABBA and BM4 use the duplicated rows. `track_energy` selects the energy variant. For N planar particles the respective dimensions are $2N$, $4N$, $4N$, and $6N$: one time entry and one momentum per particle. All time entries equal the integration time. Classical full-cyclotron runs instead use $4N$ or $6N$ entries. Accepted spatial copies coincide; only one physical copy is exported. Every coordinate block contains N values in component-major order. The formulation validates particle times and aligns output round-off; the integrator does not interpret coordinate indices.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, z), \quad \kappa(t_0) = 0, \quad \Delta K(t) = H(t, z(t)) + \kappa(t) - H(t_0, z_0).$$

Splitting accumulators from both copies are divided by two. The physical Hamiltonian may vary with time; ΔK measures its computed balance. Tracking leaves physical trajectories, nonlinear stopping rules and adaptive acceptance unchanged. Dynamics must supply the momentum derivative, including zero for an autonomous Hamiltonian.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired; `state_extension='fully_extended'` raises migration guidance. Dynamics, formulation, method and integration remain separate. The shared state classes own layouts, diagonal embedding and history extraction in

```
src/formulations/state.py
```

Each integration initializes a fresh method instance. Observers receive physical states; energy histories are in `Solution.diagnostics`. The `extended_time` history has shape (N, n_{samples}) . Fixed-step shadow samples leave main states and counters unchanged. Adaptive energy quadrature uses accepted dense output; its backend state and Jacobian remain physical.